

Control 1

Integrantes: Francisca López
Rakhel Gomez Montero
Profesor: Ivana Bachmann
Auxiliar: Julián Ferreira
Fecha de entrega: 07 de Septiembre de 2026
Santiago de Chile

Índice de Contenidos

1. Introducción	1
2. Desarrollo	2
2.1. Parte 1: Parseo y creación de mensajes HTTP	2
2.2. Parte 2: Proxy HTTP	2
2.2.1. Sockets utilizados	2
2.2.2. Recepción con buffer pequeño	2
2.2.3. Bloqueo y censura	3
2.3. Decisiones de diseño	3

1. Introducción

En el siguiente informe se presentara un problema relacionado a HTTP, este se refiere a un protocolo orientado a transacciones de tipo independientes.

Estos constan de un área de headers llamado HEAD que contiene la información necesaria sobre el mensaje como es el tipo de contenido, el largo del contenido, el navegador en uso, el host, etc. Y ademas, un área de datos llamado BODY que se refiere al área de datos respectivamente.

2. Desarrollo

2.1. Parte 1: Parseo y creación de mensajes HTTP

Se implementaron dos funciones reutilizadas en toda la actividad.

`parse_HTTP_message` recibe los bytes crudos del socket y los transforma en un diccionario. Primero separa headers y body usando `\r\n\r\n`. Luego revisa la start line: si comienza con "HTTP/" es un response (extrae versión, código y texto de estado); si no, es un request (extrae método, path y versión). En ambos casos se extraen los headers como diccionario y el body.

`create_HTTP_message` hace el proceso inverso: arma la start line (distinguiendo request de response según si el diccionario trae la llave `body`), concatena los headers, agrega el separador y el body, y devuelve bytes listos para enviar por el socket.

Usar un diccionario de headers permite agregar o modificar campos (por ejemplo `X-ElQuePregunta`).

2.2. Parte 2: Proxy HTTP

2.2.1. Sockets utilizados

Diagrama de Flujo y Sockets:

```

Cliente --(1) request-->  PROXY  --(2) request-->  Servidor
          <--(4) response--          <--(3) response--

```

- `client_socket`: entregado por `accept()` en el socket servidor del proxy. Se usa para recibir el request del navegador y para devolverle la respuesta final (pasos 1 y 4).
- `server_socket`: abierto dentro de `handle_client` con `connect()` hacia el host de destino. Se usa para reenviar el request y recibir la respuesta real (pasos 2 y 3).

El destino (host, port) se obtiene de la URI absoluta cuando el path del request comienza con `://` si no se obtiene del header `Host`.

2.2.2. Recepción con buffer pequeño

`receive_http_message` acumula lo recibido en `data` dentro de un ciclo `while True`. La decisión de cuándo parar se hace sobre todo lo acumulado, no solo el último chunk, ya que con `BUFFER = 50` tanto la start line como los headers pueden llegar repartidos entre varias llamadas a `recv`.

- **¿Mensaje completo?** Cuando ya llegó `\r\n\r\n` y el body alcanza el largo de `content_length`.
- **¿Headers no caben en el buffer?** No hay problema: la variable `headers_done` solo pasa a `True` cuando el separador aparece en `data`, así que el ciclo sigue pidiendo más `chunks` hasta entonces.
- **¿HEAD completo?** En el momento en que `b"\r\n\r\n"` aparece por primera vez dentro de `data`.
- **¿BODY completo?** Cuando `len(body)` es $\geq$ `content_length`.

2.2.3. Bloqueo y censura

En `handle_client`, el host y el path de destino se concatenan en `target` y se comparan contra `BLOCKED`, normalizando ambos lados (quitando el prefijo `"http://"` y la barra final) para que la comparación no dependa del formato exacto del JSON.

Si el destino coincide, `build_forbidden_response` arma directamente un `403 Forbidden` con un HTML que referencia `yona.jpg`, sin abrir conexión hacia el servidor real. Antes de eso, si el recurso pedido es exactamente `/yona.jpg`, el proxy responde con la imagen local sin pasar por la lógica de bloqueo.

Si el destino no está bloqueado, se agrega `X-ElQuePregunta` con `USER`, se elimina `Proxy-Connection`, se fuerza `Connection: close`, y se reenvía la petición. Al recibir la respuesta, si `Content-Type` comienza con `text/html`, se reemplaza cada palabra de `FORBIDDEN` por su valor de censura y se recalcula `Content-Length`.

2.3. Decisiones de diseño

- Headers como diccionario, para modificarlos e inyectar campos fácilmente.
- Se elimina `Proxy-Connection` antes de reenviar, ya que es específico de la comunicación cliente-proxy.
- Se fuerza `Connection: close` en la respuesta, simplificando la detección de fin de mensaje en `receive_http_message`.
- El host y el path se separan a mano con `partition`, respetando la restricción.